SWE 4739
Embedded Software Development

Lecture 4
RTOS Application Design

Sabrina Islam
Lecturer, CSE, IUT
Contact: +8801832239897
E-mail: sabrinaislam22@iut-dhaka.edu

Real-Time Operating System (RTOS)

- RTOS is much smaller than systems like Android or Windows.
- Requires only a few kilobytes of storage.
- Size depends on configuration and application needs.
- RTOS provides:
  - A multithreading environment
  - At least one scheduling algorithm
  - Mutexes, semaphores, queues, and event flags
  - Middleware components (generally optional)

Real-Time Operating System (RTOS)

Benefits

- Saves time and development effort.
- Provides ready-made tools instead of building from scratch.

Common Design Challenges

- How many tasks should the application have?
- How much work should each task do?
- Can there be too many tasks?
- How should task priorities be set?

Tasks, Threads, and Processes

Task

General OS View: A task is a concurrent and independent program that competes for execution time on a CPU.

Embedded System View: A task is a semi-independent portion of the application that carries out a specific duty.

- It is a separate “program.”
- It may interact with other tasks (programs) running on the system.
- It has a dedicated function or purpose.

Tasks, Threads, and Processes

Thread

A thread is a semi-independent program of the application that carries out a specific duty.

Form the embedded system perspective, Tasks and Threads mean the same thing.

Tasks, Threads, and Processes

Process

A process is a collection of tasks or threads and associated memory that runs in an independent memory location.

- A process is protected through a Memory Protection Unit (MPU).
- MPU controls which memory a process can access.

Tasks, Threads, and Processes

Process

Benefits:

- Resource Grouping
  - Groups related tasks and resources.
  - Helps achieve a specific application goal.
- Improved Robustness
  - If one process fails, others are protected.
- Improved Security
  - Limits what each process can access.
  - Prevents unwanted memory access.

Tasks, Threads, and Processes

RTOS objects in a single address space

Tasks, Threads, and Processes

RTOS objects into multiple, independent address spaces

Task Decomposition Techniques

Two primary task decomposition techniques:

- Feature-Based Decomposition
- The Outside-In Approach

Feature-Based Decomposition

Feature-based decomposition is the process of breaking an application into tasks based on the application features.

- Decomposing an application based on features is a straightforward process.
- A developer can start by simply listing the features in their application.
- Then convert each feature into a potential task.

Feature-Based Decomposition

Example:
Task decomposition for an IoT thermostat
Typical features might include:

- Display
- Touch screen
- LED backlight
- Cloud connectivity
- Temperature measurement
- Humidity measurement
- HVAC controller

Each feature could initially become a separate task.

Feature-Based Decomposition

Advantages

- Simple and intuitive approach.
- Easy to map system requirements to tasks.
- Good starting point for beginners using RTOS.

Disadvantages

- Can create too many tasks.
- Increases system complexity.
- May use more RAM and memory than necessary.

Optimization: Look for common functionality and combine related tasks.

Feature-Based Decomposition

The Outside-In Approach to Task Decomposition

The Seven-Step Process

The steps are relatively straightforward and include:

1. Identify the major components
2. Draw a high-level block diagram
3. Label the inputs
4. Label the outputs
5. Identify first-tier tasks
6. Determine concurrency, dependencies, and data flow
7. Identify second-tier tasks

The Outside-In Approach to Task Decomposition

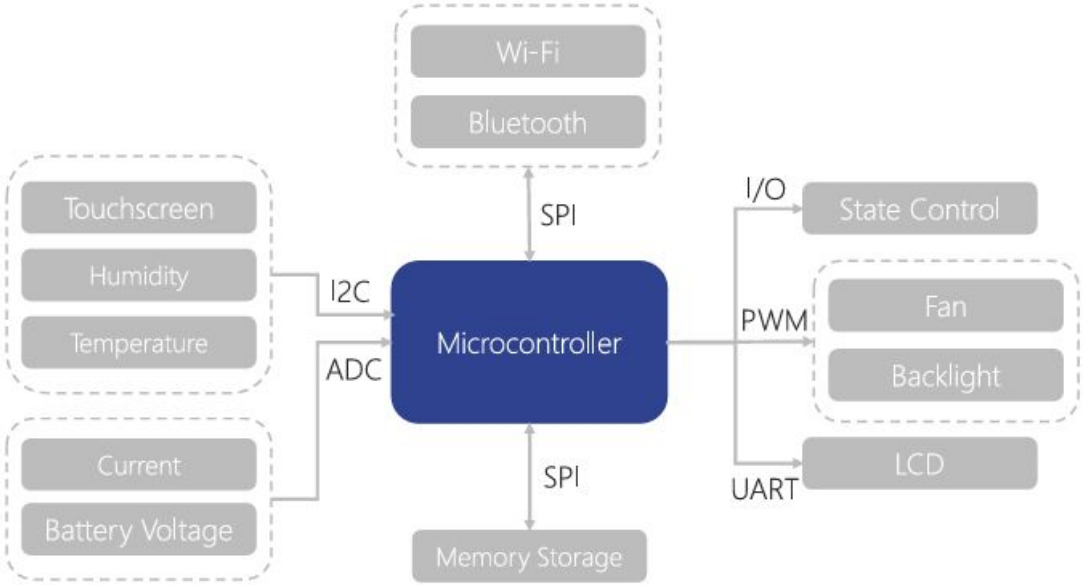
We'll see this approach through the IoT thermostat example.

An IoT thermostat has

- A wide range of sensors.
- Sensors with various filtering and sample requirements.
- Power management requirements, including battery backup management. Connectivity stacks like Wi-Fi and Bluetooth for reporting and configuration management.
- Similarities to many IoT control and connectivity applications make the example scalability to many industries.

The Outside-In Approach to Task Decomposition

The hardware block diagram for an IoT thermostat.



The Outside-In Approach to Task Decomposition

Step 1: Identify the Major Components

- Start by identifying the major hardware components that affect the software.
- These components will help determine the main tasks in the RTOS design.
- Think from the outside (hardware) and move inward toward software structure.
- It is helpful to review the schematics and hardware block diagrams to identify
- the major components contributing to the software architecture.

The Outside-In Approach to Task Decomposition

Step 1: Identify the Major Components

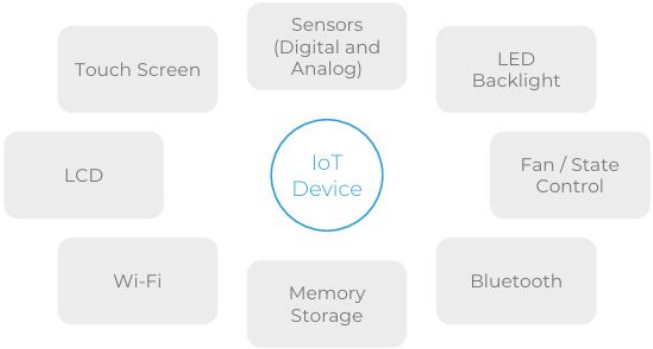
The major components of IoT thermostat:

- Humidity/temperature sensor
- Gesture sensor
- Touch screen
- Analog sensors
- Connectivity devices (Wi-Fi/Bluetooth)
- LCD/display
- Fan/motor control
- Backlight
- Etc

The Outside-In Approach to Task Decomposition

Step 2: Draw a High-Level Block Diagram

- Using the identified major components create a high-level block diagram.
- A block diagram helps us visualize how different components interact with each other and how they produce data.



The Outside-In Approach to Task Decomposition

Step 3: Label the Inputs

- "Data dictates design" principle is followed.
- After examining each major component, identify which blocks generate input data into the IoT device block.
- Also determine what the output from the block is and the input into the application.

The Outside-In Approach to Task Decomposition

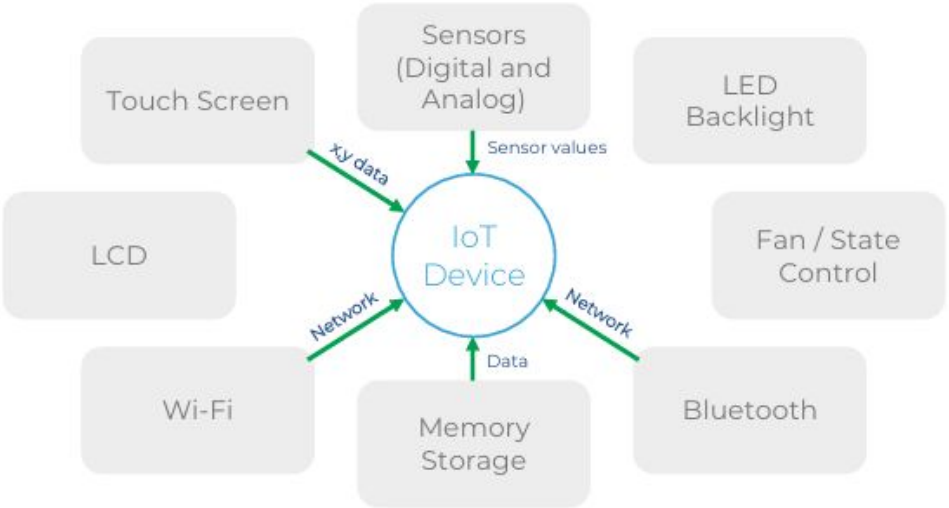
Step 3: Label the Inputs

Example:

- The touch screen will likely generate an event with x and y coordinate data when someone presses the touch screen.
- The Wi-Fi and Bluetooth blocks will generate network data.
- The sensor block will generate sensor values.

The Outside-In Approach to Task Decomposition

Step 3: Label the Inputs



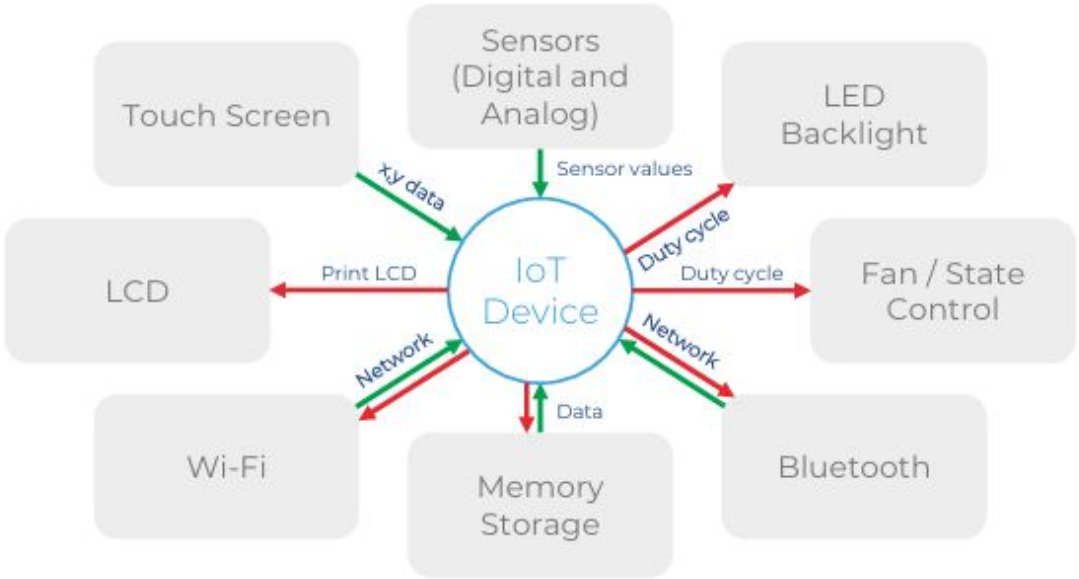
The Outside-In Approach to Task Decomposition

Step 4: Label the Outputs

- Outputs are signals/data the system sends to control the real world.
- Identify everything the device controls.
- Label the outputs on the block diagram.
- Example: The LED backlight duty cycle output data may include
  - Duty cycle 0.0–100.0%.
  - The update rate is 100 milliseconds.

The Outside-In Approach to Task Decomposition

Step 4: Label the Outputs



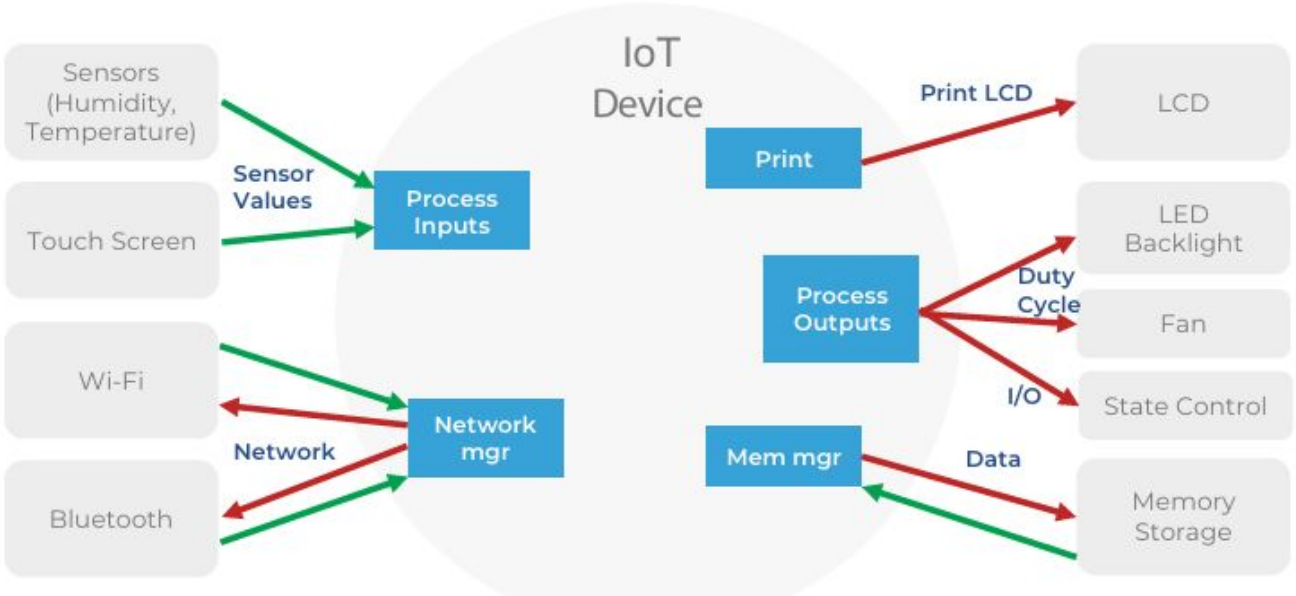
The Outside-In Approach to Task Decomposition

Step 5: Identify First-Tier Tasks

- First tasks in the system focus on interacting with the hardware components.
- The outside-in approach means starting from the “outside” (hardware) and moving inward to software tasks.
- Using the block diagram to organize tasks:
  - Look at the inputs and outputs of the system.
  - Rearrange blocks in the diagram to group related elements:
    - Sensors and touch screen grouped as sensor inputs.
    - Wi-Fi and Bluetooth grouped as connectivity blocks.

The Outside-In Approach to Task Decomposition

Step 5: Identify First-Tier Tasks

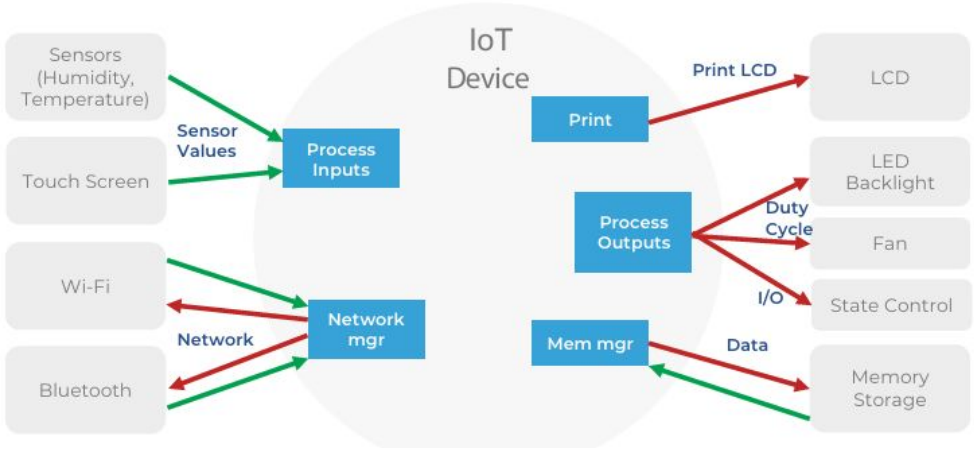


The Outside-In Approach to Task Decomposition

Step 5: Identify First-Tier Tasks

Five tasks have been identified in this initial design which include:

- Process inputs
- Network manager
- Print
- Process outputs
- Memory manager



The Outside-In Approach to Task Decomposition

Step 5: Identify First-Tier Tasks

- Task grouping:
  - Minimize the number of tasks to reduce complexity.
- Layering approach:
  - Break system into layers; each layer should contain ≤ ~12 tasks.
  - Fewer tasks per layer make the design easier to manage and less error-prone.

The Outside-In Approach to Task Decomposition

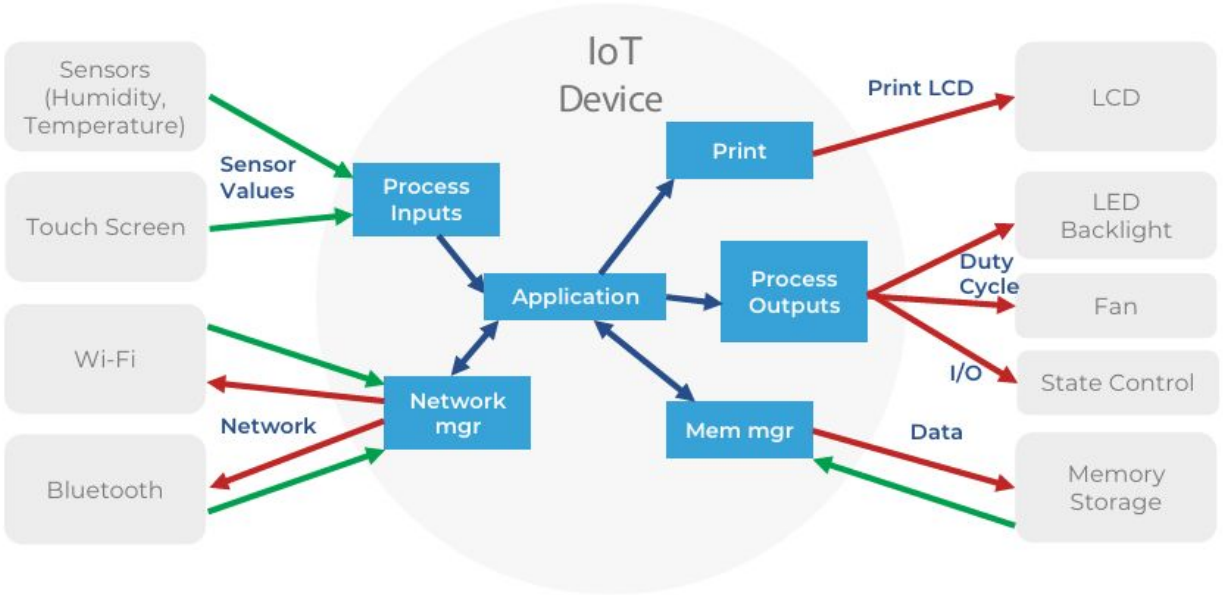
Step 6: Determine Concurrencies, Dependencies, and Data Flow

We analyze:

- How data moves between tasks
- How tasks depend on each other
- Which tasks run concurrently

The Outside-In Approach to Task Decomposition

Step 6: Determine Concurrencies, Dependencies, and Data Flow



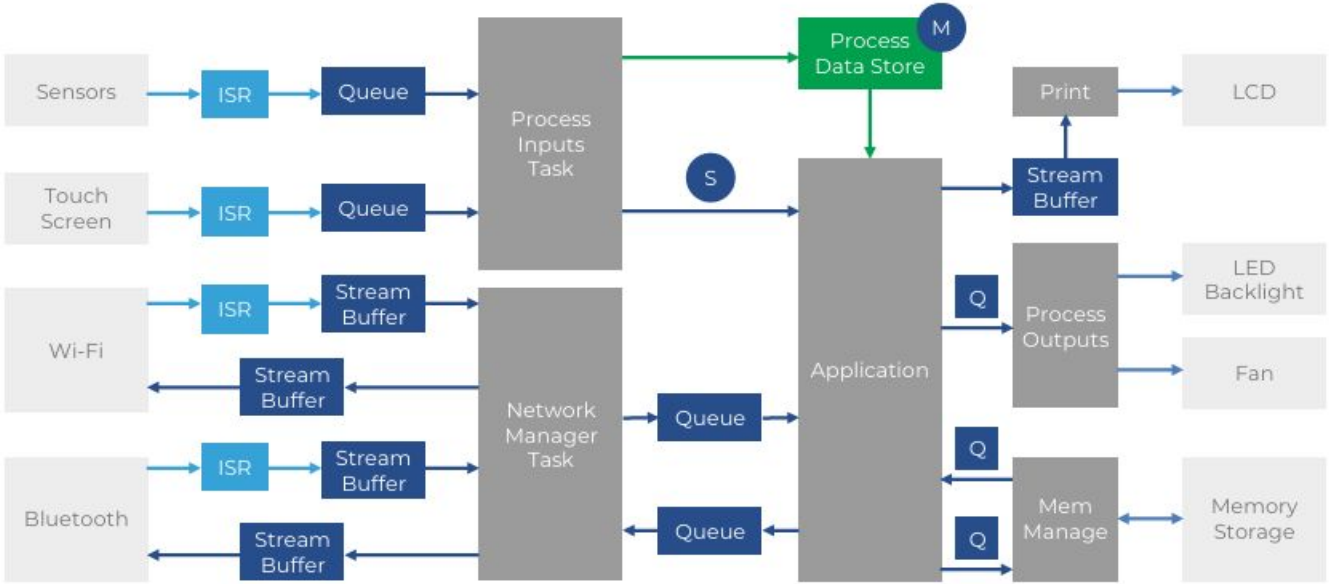
The Outside-In Approach to Task Decomposition

Step 6: Determine Concurrencies, Dependencies, and Data Flow

- Determine Concurrency
  - Tasks that run simultaneously (concurrent)
  - Tasks that depend on others
- Create a Data Flow Diagram
  - How data enters the system
  - How it moves between tasks
  - Where it is processed
  - How it exits the system

The Outside-In Approach to Task Decomposition

Step 6: Determine Concurrencies, Dependencies, and Data Flow



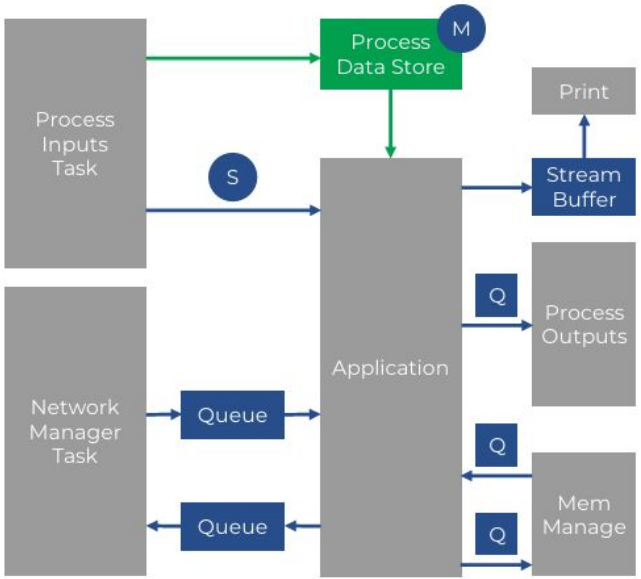
The Outside-In Approach to Task Decomposition

Step 7: Identify Second-Tier Tasks

- Take the data flow diagram.
- Remove outer hardware tasks.
- Focus on:
  - Application inputs
  - Application processing
  - Application outputs

The Outside-In Approach to Task Decomposition

Step 7: Identify Second-Tier Tasks



The Outside-In Approach to Task Decomposition

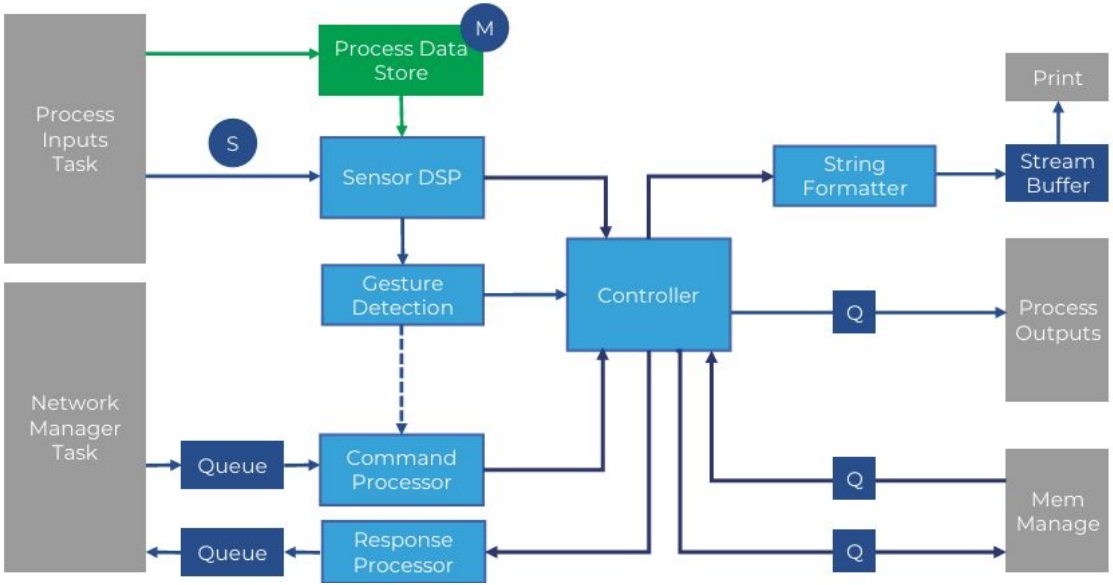
Step 7: Identify Second-Tier Tasks

For example, the sensor DSP task would behave as follows:

1. First, block execution until a semaphore is received (no new data available).
2. Then, when the semaphore is acquired, begin executing (data available).
3. Lock the sensor data mutex.
4. Access the new sensor data.
5. Unlock the sensor mutex.
6. Perform digital signal processing on the data, such as averaging or filtering.
7. Pass the processed data or results to the application task and pass the data to a gesture detection task.

The Outside-In Approach to Task Decomposition

Step 7: Identify Second-Tier Tasks



Setting Task Priorities

Task Scheduling Algorithms

- Shortest job first (SJF)
- Shortest response time (SRT)
- Rate Monotonic Scheduling (RMS)

Response time measures the total time from a request submission until the first output is produced or the task is completed from the user's perspective.

Execution time is the actual time spent by the CPU actively processing the task's instructions.

Setting Task Priorities

Shortest job first (SJF)

It is a non-preemptive scheduling algorithm where the task with the smallest total execution time (burst time) is executed first.

- All tasks are ready in the queue.
- The scheduler selects the task with the shortest CPU burst time.
- Once started, the task runs until completion.
- No interruption (non-preemptive).

Setting Task Priorities

Shortest job first (SJF)

- Advantages
  - Minimizes average waiting time
  - Efficient for batch systems
- Disadvantages
  - Requires knowing execution time in advance
  - Long tasks may suffer starvation
  - Not suitable for real-time systems

Setting Task Priorities

Shortest response time (SRT)

Shortest response time scheduling sets the task with the shortest response time as the highest priority. It is the preemptive version of SJF.

When a new task arrives:

- If its response time is shorter than the currently running task, the current task is preempted.
- The new shorter task runs immediately.

Setting Task Priorities

Shortest response time (SRT)

- Advantages
  - Very low average waiting time
  - Good responsiveness
- Disadvantages
  - More context switching overhead
  - Possible starvation of long tasks
  - Hard to predict exact execution time

Setting Task Priorities

Rate Monotonic Scheduling (RMS)

RMS is a fixed-priority real-time scheduling algorithm for periodic tasks. It sets the task priority with the shortest period.

- Advantages
  - Predictable and deterministic
  - Good for embedded and real-time systems
  - Simple fixed priority implementation
- Disadvantages
  - Not optimal for all cases
  - Lower CPU utilization compared to dynamic methods (like EDF)
  - Requires tasks to be periodic and independent

Setting Task Priorities

Rate Monotonic Scheduling (RMS)

The basic version of RMS has several critical assumptions that developers need to be aware which include

- Tasks are periodic.
- Tasks are independent.
- Preemptive scheduling is used.
- Each task has a constant execution time (can use worst case).
- Aperiodic tasks are limited to start-up and failure recovery.
- All tasks are equally critical.
- Worst-case execution time is constant.

Some of these are unrealistic for the real-world scenario.

Setting Task Priorities

Rate Monotonic Analysis(RMA)

The primary test to determine if all system tasks can be scheduled successfully is to use the basic RMA equation:

$$\sum_{k=1}^{n} \frac{E_k}{T_k} \leq n\left(2^{\frac{1}{n}} - 1\right)$$

In this equation, the variables are defined as follows:

- E_k is the worst-case execution time for the task.
- T_k is the period that the task will run at.
- n is the number of tasks in the design.

Setting Task Priorities

Rate Monotonic Analysis(RMA)

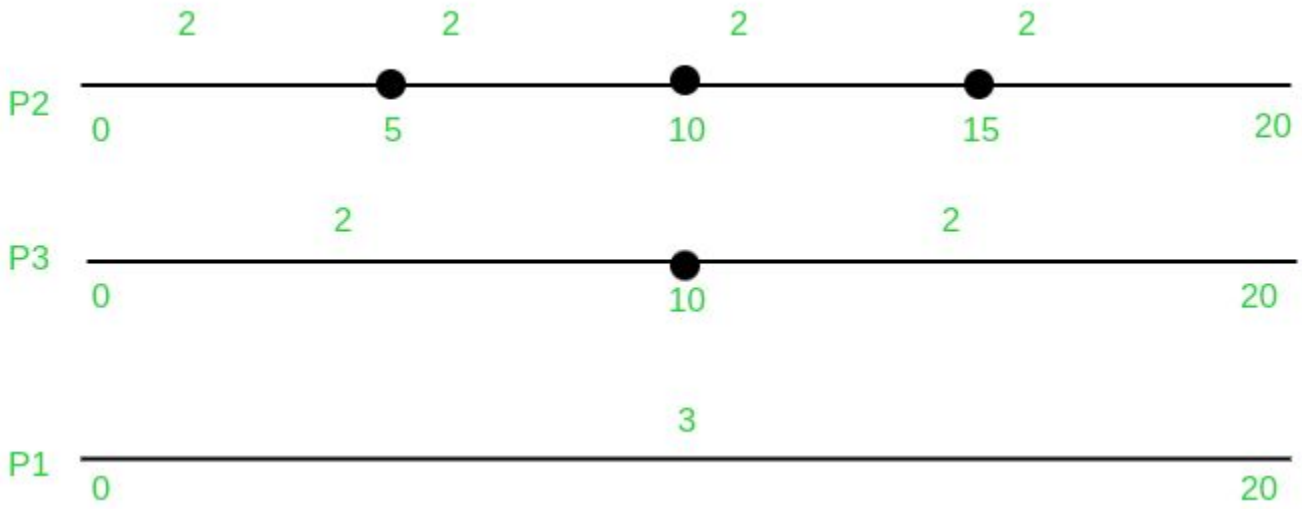
Example:

Processes	Execution Time (C)	Time period (T)
P1	3	20
P2	2	5
P3	2	10

Setting Task Priorities

Rate Monotonic Analysis(RMA)

Example:



Setting Task Priorities

Rate Monotonic Analysis(RMA)

How to do the analysis and simulate the scheduling using RMS for this example?

Setting Task Priorities

Measuring Execution Time

Rate monotonic analysis, the priorities we assign our tasks, and so forth, are very dependent upon the execution time of our tasks. There are several different techniques that can be used to measure task execution time.

- Measuring Task Execution Time Manually
- Measuring Task Execution Time Using Trace Tools

Setting Task Priorities

Measuring Task Execution Time Manually

Manual measurement means the developer adds extra code or hardware signals to measure how long a task runs.

Common Methods

1. Using GPIO Pins
    a. A GPIO pin is turned HIGH at the start of a task.
    b. The pin is turned LOW when the task finishes.
2. Using Internal Timers
    a. A timer starts when the task begins.
    b. The timer stops when the task ends.
    c. The difference gives the execution time.

Setting Task Priorities

Measuring Task Execution Time Manually

Problems with Manual Measurement

- Inconsistent Measurements
  - Developers often measure tasks at different times during development.
  - Measurements may be done only when debugging, not continuously. This leads to inconsistent timing results.
- Code Instrumentation Required
  - Developers must insert extra measurement code into the program.
- Difficult to Measure Multiple Tasks
  - In complex RTOS systems, there may be many tasks running concurrently.
  - It may not be possible to instrument all tasks at once.
  - Developers often measure one task at a time, making the process slow.

Setting Task Priorities

Measuring Task Execution Time Using Trace Tools

- Most RTOS kernels include a built-in tracing feature that records system events to analyze performance and assist debugging.
- The kernel automatically logs events such as task switches, semaphore operations, system ticks, and mutex locks/unlocks, along with their timestamps.
- These recorded events allow developers to reconstruct the order and timing of system activities, helping analyze task execution and system behavior.

Setting Task Priorities

Measuring Task Execution Time Using Trace Tools

Trace Data Collection

- A recording library stores kernel events in a RAM buffer.
- Data can be transferred in two ways:
  - Snapshot mode: buffer is read when full.
  - Streaming mode: events are continuously sent to the host in real time.
- A host-side application retrieves the data and visualizes system performance.

Reference:

1. Embedded Software Design: A Practical Approach to Architecture, Processes, and Coding Technique - Jacob Beningo

Thank You!